

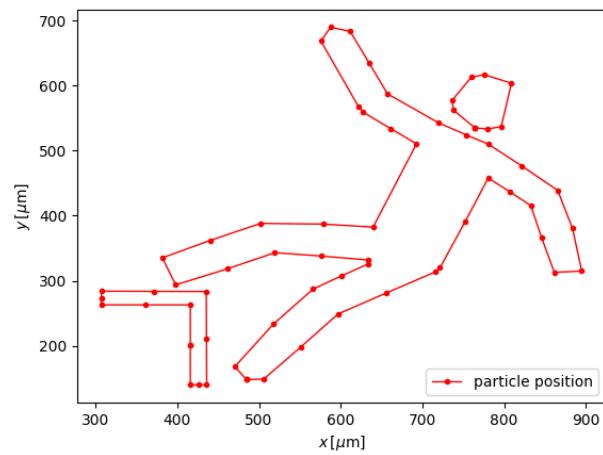


Abb. 0.1: Tracken der Position eines Brownschen Ladungsträgers³

Auswertung Versuch 243 - Bestimmung der Boltzmannkonstante mit Hilfe von thermischem Rauschen

Physikalisches Praktikum PAP 2.1

des Studienganges Physik

an der Universität Heidelberg

von

Benjamin Kleijn

12. Juli 2026

Versuchstermin 26.05.26

Tutor:in Alexios Mouratidis

Inhaltsverzeichnis

1. Einleitung	4
1.1. Motivation	4
1.2. Ziele des Versuches	4
1.3. Geräte	4
1.4. Theoretische und technische Grundlagen	5
2. Durchführung	7
2.1. Messverfahren	7
2.2. Messprotokoll	8
3. Auswertung	9
3.1. Beobachtungen Vorversuch	9
3.2. Frequenzgang	10
3.3. Bandbreite berechnen	11
3.4. Fitten der effektiven Spannungen	11
3.5. Boltzmannkonstante	12
4. Zusammenfassung und Diskussion	13
4.1. Vorversuch	13
4.2. Frequenzgang	13
4.3. Fitgüte der eff. Spannungen	14
4.4. Boltzmannkonstante	14
4.5. Fazit	14
Literaturquellen	15
Abbildungsquellen	15
A. Formeln der statistischen Analyse	16
B. Code-Anhang	16

Abbildungsverzeichnis

0.1. <i>Meme</i> : Position eines Brownschen Ladungsträgers ³	1
1.1. Versuchsaufbau/Geräte ¹	5
2.1. Schema des Versuchsaufbaus ¹	8
3.1. Rauschspektrum für $f = (100 \text{ bis } 2\,000\,000) \text{ Hz}$	9
3.2. Rauschpegel Plot (5 und 30) k Ω	9
3.3. Rauschspektrum für $f = (100 \text{ bis } 3000) \text{ Hz}$ und vorgeschaltetem Bandpassfilter	10
3.4. Fit an Frequenzgang des Bandpassfilters	11
3.5. Fit an rms-Werte der Rauschspannungen	12
4.1. Rauschspektrum für $f = (100 \text{ bis } 3000) \text{ Hz}$ und vorgeschaltetem Bandpassfilter	13

Tabellenverzeichnis

4.1. Vergleich der Boltzmannkonstante mit dem Literaturwert	14
---	----

1. Einleitung

1.1. Motivation

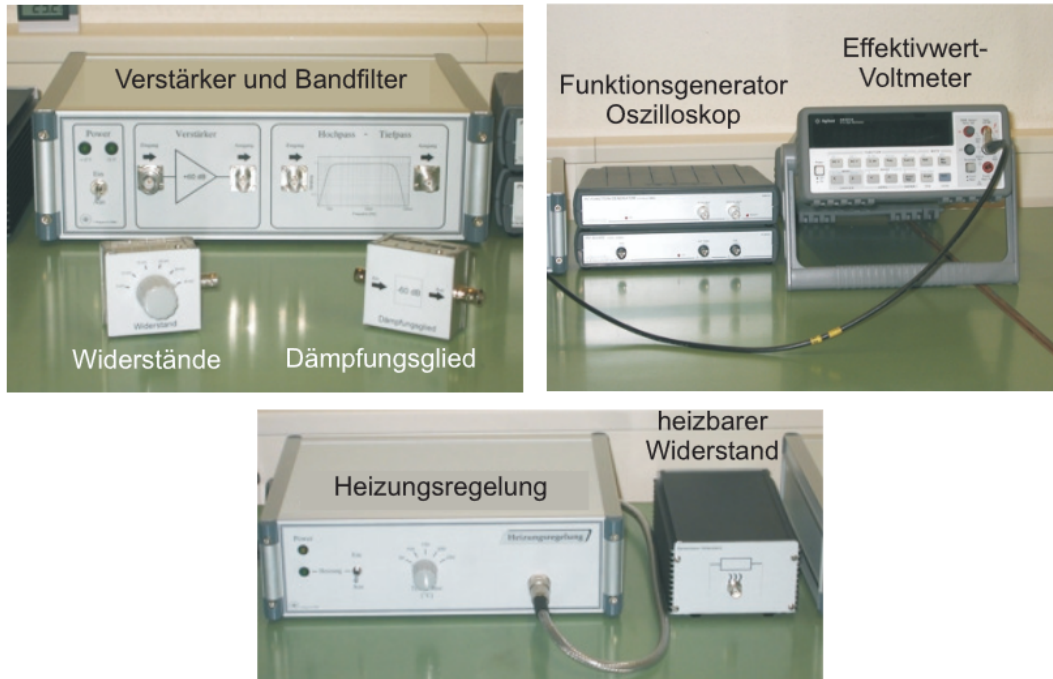
Kein Messsignal ist perfekt, sondern es ist stets mit einem Rauschen behaftet, welches die Messgenauigkeit negativ beeinflusst und begrenzt. Kenntnis des Rauschens und seiner Ursachen/Quellen erlaubt, die störenden Effekte zu minimieren und das Signal bestmöglich zu im Gegensatz zum Rauschen zu messen (Signal/Rauschverhältnis maximieren). Möglichkeiten sind zum Beispiel das Einsetzen von Kühlung (Verringerung von thermischen Rauschen), das Verringern der Messbandbreite, Mittelwertbildung, Verwendung von rauscharmen Bauteilen oder Abschirmung von Rauschquellen.

1.2. Ziele des Versuches

Aus der Bestimmung der Rauschspannung an verschiedenen Widerständen und einem Verstärker kann die Boltzmannkonstante k_B bestimmt werden. Da die Rauschspannungen im Mikrovolt-Bereich liegen, ist ein rauscharmer Verstärker notwendig, dessen Eigenrauschen separat bestimmt und vom Gesamtsignal abgezogen werden muss. Zusätzlich wird der Frequenzgang eines Verstärkers und eines Bandfilters aufgenommen, um diese in der Auswertung in Python zu untersuchen.

1.3. Geräte

1. Netzteil mit zwei 12 V Batterien
2. Zimmerthermometer
3. Verschiedene ohmsche Widerstände in einem gemeinsamen Gehäuse montiert
4. Rauscharmer Verstärker mit 1000-facher Verstärkung (60 dB) und zuschaltbarem Bandpassfilter
5. Dämpfungsglied 1/1000-fach (-60dB)
6. Beheizbarer Widerstand $4\text{ k}\Omega$ bei 0°C (Pt4000-Widerstand)
7. Heizungsnetzteil mit integrierter Regelelektronik für den beheizbaren Widerstand
8. Oszilloskop mit Spektrumanalyzer
9. Funktionsgenerator
10. Multimeter Agilent HP34401
11. PC

Abb. 1.1: Versuchsaufbau/Geräte¹

1.4. Theoretische und technische Grundlagen¹

Thermisches Rauschen, auch als Johnson-Nyquist-Rauschen bezeichnet, ist ein statistisches Phänomen, das in allen elektrischen Leitern auftritt, sofern diese eine Temperatur oberhalb des absoluten Nullpunkts besitzen. Da es ein chaotisches System ist, sind nur statistische Messungen, also über Mittlung gewonnene Ergebnisse sinnvoll. Die Ursache liegt in der thermischen Energie der Ladungsträger, die eine ungeordnete, zufällige Bewegung ausführen. Diese Bewegung wird oft als Brownsche Bewegung der Ladungsträger beschrieben. Selbst ohne Anlegung einer äußeren Spannung führt diese ungerichtete Bewegung zu einem statistisch variierenden elektrischen Potenzial im Leiter. Misst man die Spannung über einem Widerstand mit einem hochempfindlichen Oszilloskop, so beobachtet man eine fluktuierende Spannung $U_r(t)$, die statistisch um einen Mittelwert schwankt. Da sich die Anzahl der Teilchen im Widerstand proportional zur Avogadro-Konstante $N_A = 6.022 \cdot 10^{23} \frac{1}{\text{mol}}$ ist, und es somit viele zufällige Möglichkeiten (auch zeitlich zufällig) der ungerichteten Bewegung gibt, überlagern sich die Bewegungen genähert zu Wechselspannungen, die Messung detektiert also Wechselspannungen einer kontinuierlichen Frequenzverteilung. Zudem sorgt die ungerichtete Bewegung dafür, dass sich positive und negative Beiträge im zeitlichen Mittel aufheben, sodass der Erwartungswert der Rauschspannung verschwindet:

$$\langle U_r \rangle = \lim_{t' \rightarrow \infty} \frac{1}{t'} \int_0^{t'} U_r(t) dt = 0 \quad (1.1)$$

Um die Stärke des Rauschens dennoch zu quantifizieren, muss daher nicht der Mittelwert, sondern der quadratische Effektivwert (Root Mean Square oder auch rms) herangezogen werden. Alle im Versuch gemessenen Rauschspannungen sind solche rms-Werte. Dieser entspricht der Wurzel aus dem zeitlichen Mittelwert des Quadrats der Spannung:

$$\sqrt{\langle U_r^2 \rangle} = \sqrt{\lim_{t' \rightarrow \infty} \frac{1}{t'} \int_0^{t'} U_r(t)^2 dt}. \quad (1.2)$$

Bei der Untersuchung des Frequenzspektrums einer solchen thermischen Rauschquelle stellt man fest, dass alle Frequenzanteile bis in den Terahertz-Bereich in gleichem Maße vorhanden sind. Keine Frequenz wird gegenüber einer anderen bevorzugt. In Analogie zum weißen Licht, bei dem alle sichtbaren Frequenzen gleich stark vertreten sind, wird dieses Phänomen als weißes Rauschen bezeichnet. Harry Nyquist leitete 1927 eine Beziehung her, die den quadratischen Effektivwert der Rauschspannung $\langle U_r^2 \rangle$ eines ohmschen Widerstands R bei der Temperatur T in Abhängigkeit von der Messbandbreite Δf beschreibt:

$$\langle U_r^2 \rangle = 4k_B T R \Delta f. \quad (1.3)$$

In dieser Gleichung stellt k_B die gesuchte Boltzmann-Konstante dar und R die allgemeine Gaskonstante. Die Bandbreite Δf bezieht sich hierbei auf die Bandbreite der verwendeten Messelektronik (Verstärker, Filter, Messgerät). Da das Rauschen frequenzunabhängig ist, trägt jeder Frequenzbereich gleichermaßen zur gesamten Rauschleistung bei; eine größere Bandbreite führt somit zu einer höheren gemessenen Rauschspannung.

Da die Rauschspannung bei Zimmertemperatur und typischen Widerstandswerten und Bandbreiten nur im Bereich von Mikrovolt liegt, ist eine direkte Messung ohne Verstärkung nicht möglich. Deshalb wird ein rauscharmer Verstärker mit einer Verstärkung V (im Versuch ca. 1000-fach bzw. 60 dB) eingesetzt. Allerdings stellt der Verstärker selbst eine weitere Rauschquelle dar. Die Rauschbeiträge des Widerstands $\langle U_R^2 \rangle$ und des Verstärkers $\langle U_V^2 \rangle$ addieren sich quadratisch, da sie statistisch unabhängig voneinander sind und ihre Mittelwerte null sind:

$$\langle U_{R+V}^2 \rangle = \langle (U_R + U_V)^2 \rangle = \langle U_R^2 \rangle + \langle U_V^2 \rangle. \quad (1.4)$$

Um die Boltzmann-Konstante bestimmen zu können, muss das Eigenrauschen des Verstärkers $\langle U_V^2 \rangle$ experimentell ermittelt werden, indem der Verstärkereingang kurzgeschlossen wird (Nullmessung), und später von der Gesamtmessung subtrahiert werden. Zudem wird zur präzisen Bestimmung der effektiven Bandbreite ein Bandfilter verwendet, welcher aus einem Hochpass- und einem Tiefpassfilter besteht. Der Hochpass unterdrückt niederfrequente Störungen, während der Tiefpass hochfrequentes Rauschen filtert und eine scharfe Begrenzung der oberen Grenzfrequenz ermöglicht.

Zur Bestimmung der Bandbreite wird der Frequenzgang $g(f)$ des gesamten Messsystems (Verstärker plus Bandfilter) untersucht, dieser ist definiert als das Verhältnis der Ausgangsspannung zur Eingangsspannung bei einer bestimmten Frequenz f :

$$g(f) = \left. \frac{U_{\text{aus}}(f)}{U_{\text{ein}}(f)} \right|_f. \quad (1.5)$$

Die Ausgangsrauschleistung ergibt sich durch Integration des quadratischen Frequenzgangs über alle Frequenzen, multipliziert mit der Nyquist-Leistungsdichte. Unter Berücksichtigung des Verstärkerrauschens gilt für die gemessene Ausgangsspannung:

$$\langle U_{\text{aus}}^2 \rangle = 4k_B T R \int_0^\infty g(f)^2 df + \langle U_V^2 \rangle = 4k_B T R B + \langle U_V^2 \rangle. \quad (1.6)$$

Das Integral $B = \int_0^\infty g(f)^2 df$ wird als äquivalente Rauschbandbreite des Messsystems bezeichnet. Durch Umstellen dieser Gleichung lässt sich die Boltzmann-Konstante berechnen:

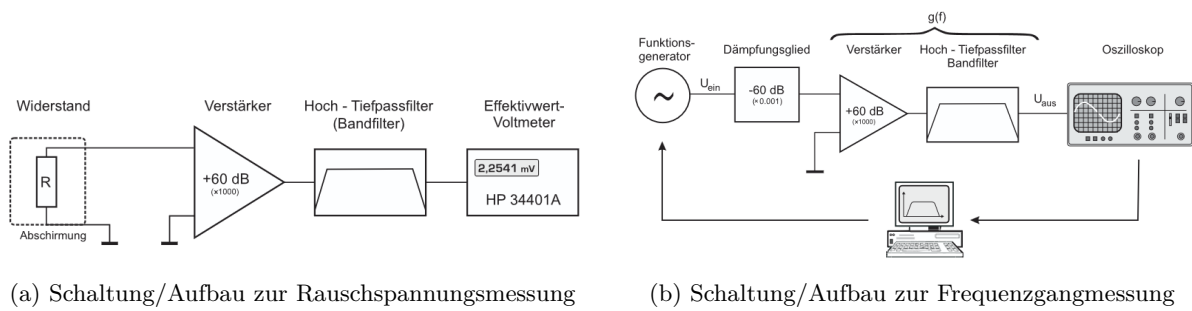
$$k_B = \frac{\langle U_{\text{aus}}^2 \rangle - \langle U_V^2 \rangle}{4TRB}. \quad (1.7)$$

2. Durchführung

2.1. Messverfahren

Der erste Versuchsaufbau in Abb. 2.1a dient der Rauschmessung der Rauschquelle, die aus verschiedenen ohmschen Widerständen in einem abgeschirmten Gehäuse besteht. Diese werden direkt an den Eingang eines rauscharmen Verstärkers angeschlossen. Der Ausgang des Verstärkers ist mit einem Bandfilter verbunden, welches die gewünschte Messbandbreite definiert und Störsignale filtert. Das gefilterte Signal wird schließlich an ein hochempfindliches Effektivwert-Multimeter oder ein Oszilloskop geleitet, um die Rauschspannung zu messen. Nach Starten des Analyseprogramms am PC werden damit 100 Einzelmessungen (Spannungswert des Voltmeters wird vom Programm erkannt) vorgenommen, deren Mittelwert und Standardabweichung automatisch berechnet und in einer `.txt` Datei gespeichert. Selbiges Vorgehen erfolgt zur Messung des Eigenrauschens des Verstärkers mit dem Kurzschlussstecker.

Zur Bestimmung des Frequenzgangs wird ein Funktionsgenerator verwendet, der ein Sinussignal ausgibt. Dieses Signal wird vor dem Verstärker durch ein Dämpfungsglied (Dämpfungsfaktor $D = 10^{-3}$) stark abgeschwächt, um eine Übersteuerung (Sättigung) des Verstärkers zu vermeiden. Das Ausgangssignal des Bandfilters wird hierbei mit einem Oszilloskop gemessen. Die gesamte Steuerung der Messgeräte sowie die Datenerfassung erfolgen über einen PC, der Aufbau

Abb. 2.1: Schema des Versuchsaufbaus¹

ist in Abb. 2.1b zu sehen. Die Daten werden vom Messprogramm automatisch als `.csv` Datei gespeichert.

2.2. Messprotokoll

Es existiert kein Messprotokoll, die Daten wurden von einem auf dem PC laufenden Python-Programm namens *waveforms* automatisch als `.txt` bzw. `.csv` Datei gespeichert.

3. Auswertung

3.1. Beobachtungen Vorversuch

Im Vorversuch wurden Rauschspektren aufgenommen, d.h. erfasst, mit welchem Pegel Frequenzen im Bereich von (100 bis 2 000 000) Hz im Rauschsignal vorhanden sind. Dazu wurden die Widerstände von (5 bis 30) k Ω direkt ans Oszilloskop/PC angeschlossen, eine weitere Aufnahme wurde bei einem erhöhten Frequenzbereich auf 3 MHz und bei vorgeschaltetem Bandfilter gemacht. Die Spektren sind in den folgenden Abbildungen zu sehen.

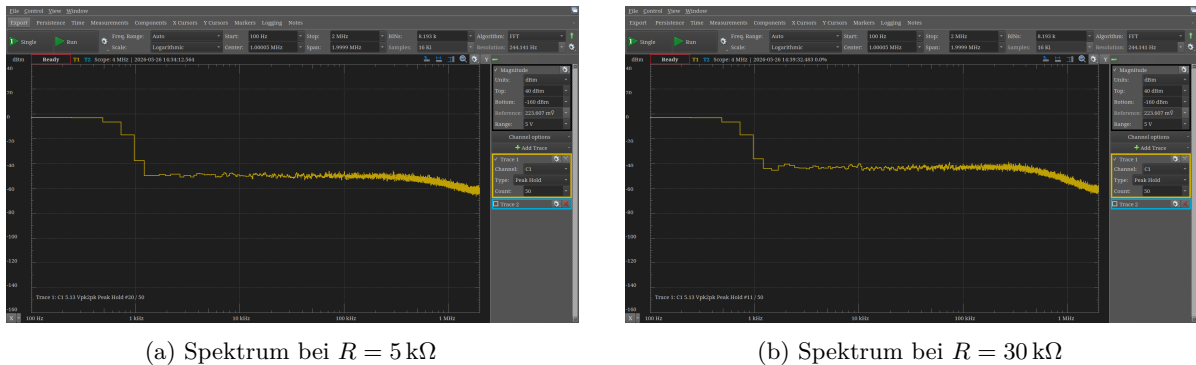


Abb. 3.1: Rauschspektrum für $f = (100 \text{ bis } 2\,000\,000) \text{ Hz}$

Man sieht nur sehr schwierig die Aussage der Messung: Höherer Widerstand bedeutet höherer Rauschpegel. Um es besser zu sehen, habe ich es in den selben Graphen reingeplottet. Man sieht in folgendem Bild gut die Beobachtung, die im Vorversuch gemacht werden sollte: Der

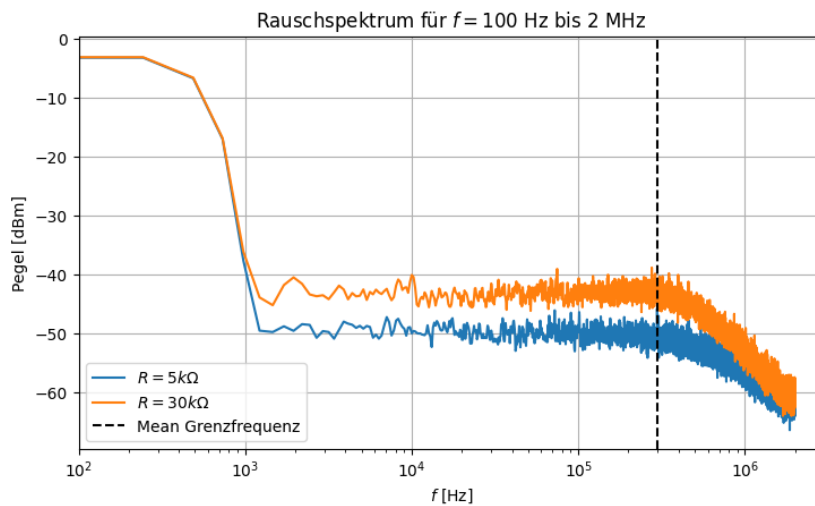


Abb. 3.2: Rauschpegel Plot (5 und 30) k Ω

Pegelunterschied im Intervall 100 Hz - 2 MHz beträgt $\Delta P = -7 \text{ dB}$, damit ist der Rauschpegel des größeren Widerstands um 224 % stärker.

Als weiterer Vorversuch sollte das erweiterte Rauschspektrum sowohl mit als auch ohne nach dem Verstärker geschalteten Bandfilter betrachtet werden:

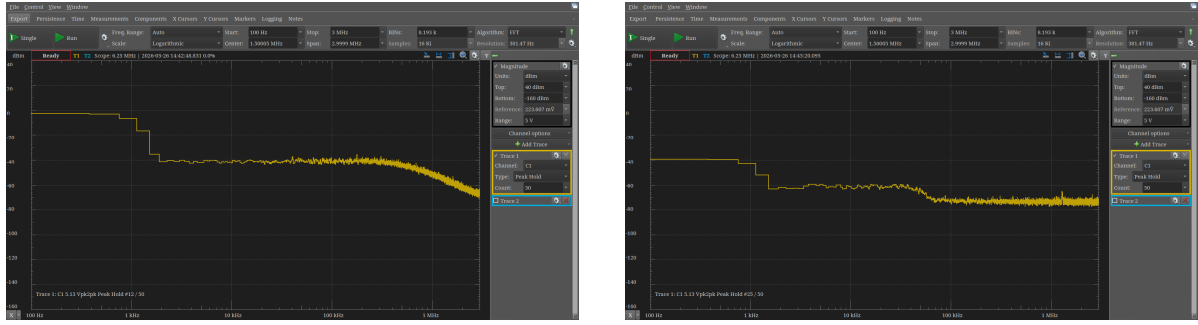
(a) $R = 30 \text{ k}\Omega$ (b) $R = 30 \text{ k}\Omega$ mit Bandpassfilter

Abb. 3.3: Rauschspektrum für $f = (100 \text{ bis } 3000) \text{ Hz}$ und vorgeschaltetem Bandpassfilter

Man erkennt gut am erweiterten Spektrum, dass der Rauschpegel für höhere Frequenzen weiterhin fällt. Dies liegt an der frequenzabhängigen Verstärkung durch den Operationsverstärker. Leider wurden zu diesen Messungen keine `.csv` Dateien abgespeichert, deswegen kann kein Plot erstellt werden und man ist auf die Beobachtungen in Abb. 4.1 eingeschränkt: Das gesamte Spektrum ist gedämpft, zudem sieht man den Effekt des Bandpassfilters oberhalb von 40 kHz : Diese Frequenzen werden noch stärker gedämpft, der Bandpass „filtert“ sie dadurch heraus.

3.2. Frequenzgang

Die aufgenommenen Werte $P(f)$ werden durch Umrechnen mit dem Dämpfungswert $D = 1/1000$ gemäß $g(f) = \frac{P}{D}$ dargestellt. An diese Messdaten soll ein Fit gemäß des Frequenzgangs $g(f)$ eines Verstärkers mit Hochpassfilter und nachgeschaltetem Tiefpass n-ter Ordnung gesetzt werden:

$$g(f) = \frac{V}{\sqrt{1 + 1/(f/\Omega_1)^{2n_1}} \sqrt{1 + 1/(f/\Omega_2)^{2n_2}}} \quad (3.1)$$

Die wichtigen Größen sind: Die hier als frequenzunabhängig genäherte Verstärkung $V = 1000$, die Grenzfrequenz des Hoch- bzw. Tiefpassfilters $\Omega_1 = 1000 \text{ Hz}$, $\Omega_2 = 50\,000 \text{ Hz}$ und die Filterordnungen $n_1 = n_2 = 5$. Als Fitergebnisse ergeben sich:

$$\begin{array}{llll} V = 1010.1 \pm 0.4 & \Omega_1 & = (1024.2 \pm 0.8) \text{ Hz} & \Omega_2 = (47\,190 \pm 40) \text{ Hz} \\ n_1 = 4.896 \pm 0.019 & n_2 & = 4.861 \pm 0.019 & \end{array}$$

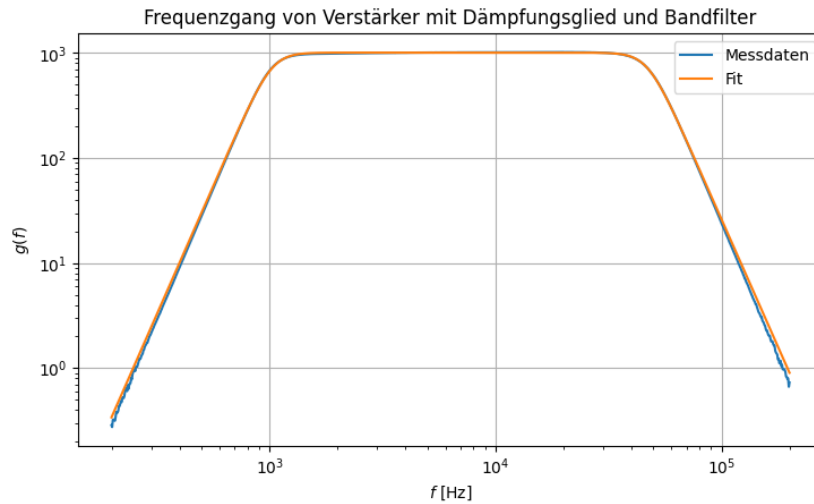


Abb. 3.4: Fit an Frequenzgang des Bandpassfilters

3.3. Bandbreite berechnen

Die angefitte Funktion kann nun benutzt werden, um die Bandbreite B , also das Integral

$$B = \int_0^\infty g(f)^2 df \quad B = (4.79 \pm 0.10) \cdot 10^{10} \text{ Hz} \quad (3.2)$$

zu berechnen. Dazu wird der quadratische Frequenzgang $g(f)^2$ aus den Fitparametern berechnet und numerisch integriert. Dies geschieht mit der Funktion `quad` aus dem Python-Paket `scipy.integrate`, die eine numerische Integration durchführt. Da der numerische Fehler sehr gering ist, der Fit nicht exakt ist (der Fitfehler ist jedoch auf jeden Fall sehr gering, da die relativen Abweichungen aller Fitparameter $< 0.4\%$), wurde der relative Fehler auf $\frac{\Delta B}{B} = 2\%$ gesetzt.

3.4. Fitten der effektiven Spannungen

Aus Aufgabe 2 im Messprotokoll können die gemessenen Spannungen $\sqrt{\langle U_{\text{aus}}^2 \rangle}$, nach quadrieren und bereinigen um die mittlere Verstärkerspannung $\langle U_V^2 \rangle$, nun abhängig vom Widerstand R gefittet werden, um somit für Gleichung (1.7) einen Wert für die Steigung c zu erhalten:

$$c = 4k_B T B = (8.088 \pm 0.019) \cdot 10^{-10} \frac{\text{V}^2}{\Omega} \quad (3.3)$$

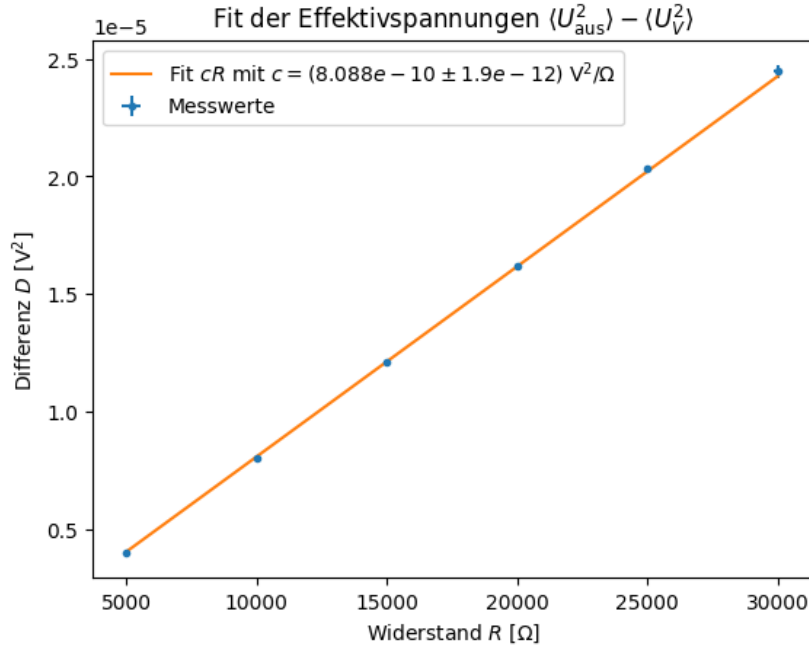


Abb. 3.5: Fit an bereinigte rms-Werte der Rauschspannungen

3.5. Boltzmannkonstante

Im Messprotokoll $T = (298.2 \pm 0.5) \text{ K}$ notiert, aus den vorherigen Berechnungen hat man B und c erhalten. Damit gilt nun

$$k = \frac{c}{4TB} \quad \Delta_{\text{stat}} k = k \sqrt{\frac{\Delta_T^2}{T^2} + \frac{\Delta_c^2}{c^2}} \quad \Delta_{\text{syst}} k = k \frac{\Delta B}{B} \quad (3.4)$$

$$k_B = (1.415 \pm 0.005 \pm 0.030) \frac{\text{J}}{\text{K}}$$

Dabei ist der erste Fehler der statistische Fehler aus der Temperatur T und der Steigung c , und der zweite Fehler der systematische Fehler aus der Bandbreite B .

4. Zusammenfassung und Diskussion

4.1. Vorversuch

Der Effekt des Bandfilters konnte nur beim $30\text{ k}\Omega$ Widerstand festgestellt werden, beim $5\text{ k}\Omega$ war keine Auswirkung sichtbar, wie man an folgenden Diagrammen sieht. Sowohl in Abb. 3.3 als auch in Abb. 4.1 sieht man, dass hohe Frequenzen in der Bandpassschaltung keinen Abfall zeigen, wie er in der bandpasslosen Schaltung aufgrund der verringerten Verstärkung hoher Frequenzen zu sehen ist. Dies liegt daran, dass der Bandpassfilter alle Frequenzen außerhalb des Filterbandes auf einen konstanten Pegel herunterdämpft. Interessant ist die Beobachtung, dass keinerlei Abfall bei niedrigen Frequenzen festgestellt werden konnte, es wäre gut zu wissen, was für ein Filterband den Bandpassfilter auszeichnet.

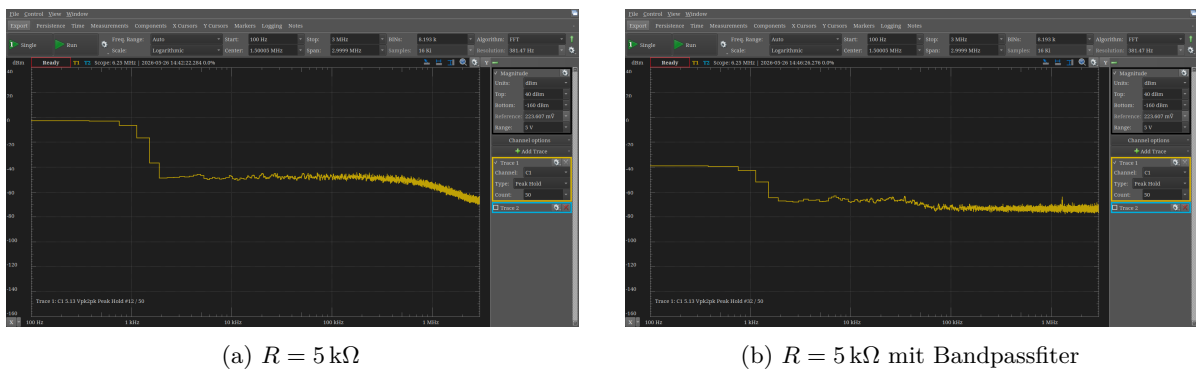


Abb. 4.1: Rauschspektrum für $f = (100 \text{ bis } 3000)\text{ Hz}$ und vorgeschaltetem Bandpassfilter

Rauschpegel: Es hat ewig gedauert, Abb. 3.2 zu erstellen, denn das Oszi-Programm hat als Spaltentrenner Kommas benutzt, aber angefangen, als Tausendertrennzeichen auch Kommas zu nutzen. Ich hab jetzt ewig gebraucht um Regex Pattern zu schreiben, die das aufheben. (war halt auch am Samstag stuck in der Hitze am Stuttgarter Hbf, kein Wunder, dass ich da nicht schneller ne Lösung gefunden habe).

Die qualitativen Beobachtungen des Rauschpegels entsprachen im Grunde der theoretischen Erwartung: Die Nyquist-Beziehung beagt, dass die Effektivspannungen (das beobachtete Rauschen) mit erhöhten Widerständen steigen, dies sieht man in Abb. 3.2.

4.2. Frequenzgang

Die Übereinstimmung zwischen Fit und Messwerten ist grandios (zumindest sagt mein Auge das). Da keine Fehler für $P(f)$ aus Abb. 3.4 existieren, kann hier leider keine χ^2 Summe berechnet werden.

4.3. Fitgüte der eff. Spannungen

Bei Evaluieren der Fitgüte von Abb. 3.5 erhält man:

$$\chi^2 = 149$$

$$\chi_{\text{red}}^2 = 29.8$$

$$p = 0 \%$$

Diese Werte sehen sehr schlecht aus. Der Grund ist, dass durch Wählen von $\Delta\sqrt{\langle U_{\text{aus}}^2 \rangle} = \frac{\sigma}{\sqrt{100}}$ mit σ der vom Programm angezeigten Standardabweichung der 100 Messwerte, der Fehler ΔD sehr klein wird. Dadurch ist χ^2 sehr hoch.

Unter Verwendung der reinen Standardabweichung als Fehler erhält man diese Werte:

$$\chi^2 = 1.49$$

$$\chi_{\text{red}}^2 = 0.29$$

$$p = 91 \%$$

Da uns immer noch nicht erklärt wurde, wie man diese Werte interpretiert, kann ich dazu nur eine Aussage treffen: Da die Fitwahrscheinlichkeit angibt, wie wahrscheinlich bei einer Wiederholungsmessung $\chi'^2 \geq \chi^2$ ist, sollte optimalerweise $p = 50 \%$ sein, denn dann tritt das Gegenteil in der anderen Hälfte der Fälle ein. Damit ist der Fit der wahrscheinlichste Fall.

4.4. Boltzmannkonstante

Beim Vergleich des bestimmten Wertes (und dem systematischen Fehler) mit dem Literaturwert der Boltzmannkonstante² ergeben sich folgende Werte: Die Sigma-Abweichung ist sehr gut.

Tabelle 4.1: Vergleich der Boltzmannkonstante mit dem Literaturwert

$k_{\text{exp}}[10^{-23} \frac{\text{J}}{\text{K}}]$	$k_{\text{lit}}[10^{-23} \frac{\text{J}}{\text{K}}]$	abs.Abw. $[10^{-23} \frac{\text{J}}{\text{K}}]$	proz.Abw. $[\%]$	$S[\sigma]$
1.42 ± 0.03	1.380649	0.03 ± 0.03	2.4 ± 2.2	1.2

Mögliche Fehlerquellen ist der (mit meinem Wissen) nicht quantifizierbare Fehler der Bandbreite B , die durch Integration des Fits bestimmt wurde, eine möglich schwankende Temperatur und der Fitwert c als Geradensteigung aus Abb. 3.5. Dieser Fit spuckt zwar fragwürdige χ^2 Werte aus, wie vorhin erwähnt, ein klarer linearer Zusammenhang ist jedoch ersichtlich.

4.5. Fazit

Guter Versuch, Rauschen zu untersuchen ist immer sehr spannend. Auch den Namen Nyquist zu lesen, auch sehr toll. Die Versuchsergebnisse zeigen keine Probleme, es existieren keine Messwerte oder Ergebnisse, die nicht erklärbar sind.

Literaturquellen

¹Dr. J. Wagner, *Physikalisches Praktikum 2.1 für Studierende der Physik B.Sc.* [Online; Stand 03/2026] (Universität Heidelberg, März. 2026), S. 33–45, https://git.physi.uni-heidelberg.de/schwierz/Versuchsanleitungen/src/branch/main/PAP2_1.pdf (besucht am 11.03.2026) (siehe S. 3, 5, 8, 16).

²Wikipedia, *Boltzmannkonstante*, [Online; Stand 05/2026], <https://de.wikipedia.org/wiki/Boltzmann-Konstante> (siehe S. 14).

Abbildungsquellen

³Wikipedia, *ISO 7010 W008*, [Online; Stand 03/2026], https://commons.wikimedia.org/wiki/File:ISO_7010_W008.svg (siehe S. 1).

A. Formeln der statistischen Analyse

Für die statistische Auswertung von n Messwerten x_i werden folgende Größen definiert [1]:

$$\text{Arithmetisches Mittel} \quad \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad (\text{A.1})$$

$$\text{Varianz} \quad \sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 \quad (\text{A.2})$$

$$\text{Standardabweichung} \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2} \quad (\text{A.3})$$

$$\text{Fehler des Mittelwerts} \quad \Delta \bar{x} = \frac{\sigma}{\sqrt{n-1}} \quad (\text{A.4})$$

$$\text{Gauß'sche Fehlerfortpfl.} \quad \Delta f = \sqrt{\sum_{i=1}^N \left(\frac{\partial f}{\partial x_i} \Delta x_i \right)^2} \quad (\text{A.5})$$

$$\text{relativer Fehler } f = xy / f = x/y \quad \frac{\Delta f}{|f|} = \sqrt{\left(\frac{\Delta x}{x} \right)^2 + \left(\frac{\Delta y}{y} \right)^2} \quad (\text{A.6})$$

$$\text{Absolute Abweichung} \quad A = |a_{lit} - a_{gem}| \quad \Delta A = \sqrt{(\Delta a_{lit})^2 + (\Delta a_{gem})^2} \quad (\text{A.7})$$

$$\text{Relative Abweichung} \quad R = \frac{A}{a_{lit}} \quad (\text{A.8})$$

$$\text{Signifikanz} \quad S = \frac{A}{\Delta A} = \frac{|a_{lit} - a_{gem}|}{\sqrt{\Delta a_{lit}^2 + \Delta a_{gem}^2}} \quad (\text{A.9})$$

Für einen Fit mit Funktionswerten y_i der Messwerte x_i mit Fehler Δx_i sowie Freiheitsgraden $f = \#(\text{Messwerte}) - \#(\text{Fitparameter})$ kann die Güte des Fits mit diesen Größen analysiert werden:

$$\chi^2 \text{ Summe} \quad \chi^2 = \sum_i \left(\frac{y_i - x_i}{\Delta x_i} \right)^2 \quad (\text{A.10})$$

$$\text{normierte reduzierte } \chi^2 \text{ Summe} \quad \chi_{\text{red}}^2 = \frac{\chi^2}{f} \quad (\text{A.11})$$

$$\text{Fitwahrscheinlichkeit} \quad p = 1 - \text{chi2.cdf}(\chi^2, f) \quad (\text{A.12})$$

Hierbei wird das Modul `chi2` von `scipy.stats` benutzt.

B. Code-Anhang

Bei den Funktionen habe ich teilweise Dinge von verschiedenen Altprotokollen zusammengesucht, und auf meine Bedürfnisse umgeschrieben. Die konkreten Berechnungen sind dann eigenständig.

July 3, 2026

```
[1]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, ROUND_HALF_UP, getcontext #_
↳ Besonders für sig. Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as c
from scipy.stats import norm
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial
import scipy.integrate as integrate

%matplotlib widget
import matplotlib.pyplot as plt
from matplotlib import colormaps
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path
```

```

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
import csv
import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

import textwrap

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

g=9.80984
Delta_g=0.00002

```

```

[2]: currentversuch = "243"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
↳ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Messwerte\243

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Auswertungen2.2\Code\243\Diagramme

Delete UTF8 False encoding

```

[3]: # def deletevocals(text):
#     text=textwrap.dedent(text)
#     patterna = r'[][a] '
#     text=re.sub(patterna,"ä",text)
#     patternu=r'[][u] '
#     text=re.sub(patternu,"ü",text)
#     patterno=r'[][o] '
#     text=re.sub(patterno,"ö",text)
#     return text

```

```

# Regexpwissen: [0-9], [A-Z] findet egal welchen Zahl/Buchstaben; [^0-9] findet
↳ erstes Zeichen, das keine Zahl ist;

def deletevowels(text):
    # Einrückungen entfernen (gemeinsame Tabs des gesamten Textes)
    text = textwrap.dedent(text)
    # entfernt Bindestriche: - findet Bindestrich, \n Zeilenumbruch direkt
↳ dahinter, \s Whitespaces *beliebig viele
    text = re.sub(r'[-\n\s*]', '', text)
    # entfernt Zeilenumbrüche
    text = text.replace("\n", " ")

    # Eine Funktion, die vom re.sub aufgerufen wird, wenn ein Treffer erzielt
↳ wird
    def convert(match):
        # match.group(1) ist der Buchstabe nach dem Pünktchen (z.B. 'a' oder
↳ 'A')
        buchstabe = match.group(1)
        # Dictionary für die echten Umlaute
        umlaute = {'a': 'ä', 'u': 'ü', 'o': 'ö', 'A': 'Ä', 'U': 'Ü', 'O': 'Ö'}
        # Gib den Umlaut zurück. Falls das Zeichen nicht im Dict ist, lass es
↳ wie es ist
        return umlaute.get(buchstabe, buchstabe)
    # Das Pattern sucht nach " gefolgt von einem beliebigen Buchstaben aus der
↳ Auswahl, [X] findet genau ein Zeichen aus Menge X
    return re.sub(r'"([auoAUO])"', convert, text)

```

```

[67]: print(deletevowels("""
Die qualitativen Beobachtungen des Rauschens entsprachen im Grunde der
↳ theoretischen Erwartung:
Einerseits besagt die Nyquist-Beziehung gerade, dass die Effektivspannungen
↳ (das beobachtete Rau-
schen) mit erhöhten Widerst"anden steige
"""))

```

Die qualitativen Beobachtungen des Rauschens entsprachen im Grunde der theoretischen Erwartung: Einerseits besagt die Nyquist-Beziehung gerade, dass die Effektivspannungen (das beobachtete Rauschen) mit erhöhten Widerständen steige

Runden signifikanter Stellen

```

[5]: def round_to_sigs(val, errVal, einheiten):
    """
    Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
↳ Diese Funktion wurde etwas umständlicher

```

geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
↳ Daher musste hier mit dezimal gearbeitet werden.

Zudem sollten besonders kleine und große Messwerte in der
↳ Dezimalschreibweise geschrieben werden, damit diese auch
für Protokolle geeignet sind.

Parameter

****val**** : float
Messwert

****errVal**** : float
Ungenauigkeit des Messwertes

Return

****value_rounded**** : str
Gerundeter Messwert

****error_rounded**** : str
Gerundete Ungenauigkeit des Messwertes

****res**** : str
"Messwert \pm Fehler"
"""

Daten zu Dezimal wechseln, da Floats probleme machen

val = Decimal(str(val))

errVal = Decimal(str(errVal))

exp = int(math.floor(math.log10(float(errVal)))) # Die erste
↳ signifikante Stellenposition wird anhand des errVal bestimmt

if round(float(errVal / (Decimal(10) ** exp))) < 3: # wenn 1,2,3
↳ erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition
↳ hinzu

exp -= 1

scale = Decimal(10) ** exp

val_round = (val / scale).quantize(Decimal('1'), rounding=ROUND_HALF_UP) *
↳ scale # mit scale wird das Komma so verschoben, dass alle
↳ signifikanten Stellen vor dem Komma stehen, und dann alle Nachkommastellen
↳ weggerundet werden

err_round = (errVal / scale).quantize(Decimal('1'), rounding=ROUND_CEILING)
↳ * scale

```

qty = f"\\qty{{{val_round}}({err_round})}{{{einheiten}}}"
num = f"\\num{{{val_round}}({err_round})}"
return float(val_round), float(err_round), num, qty

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object, object],
↳excluded=['einheiten'])

# wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
↳gewollt sind, die 10~/e Präfixe beinhalten)

```

Gauss'sche Fehlerfortpflanzung

```

[6]: def gff(function, errPronePar, latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    -----
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
    ↳ungleich den Werten für x, y, z.
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
    ↳genutzt und für deren Fehler err_x, err_y, err_z etc.

    Return
    -----
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParameters** : array
        Liste aller Fehlerbehafteten Größen
    """

    error = 0
    errProneParameters = []

    function = sp.sympify(function)
    variables = errPronePar.split(",")
    variables = sp.symbols(variables)

```

```

for variable in variables:
    Delta = sp.Symbol(f"Delta_{variable}")
    partial = sp.diff(function, variable)           # Die Funktion
    ↪ wird nach der fehlerbehafteten Variable abgeleitet
    term=sp.UnevaluatedExpr(partial*Delta)**2
    error = error + ((term))                       # Fehler werden
    ↪ quadratisch aufsummiert
    errProneParamaters.append((variable,Delta))

absolute_err=sp.simplify(sp.sqrt(error),rational = True)
relative_err=sp.simplify(sp.sqrt(error/function**2),rational = True)

if latex:
    #Prints out the Latex Code for the Functions
    print("Funktion:")
    display(Math(sp.latex(function,long_frac_ratio=2)))
    print(sp.latex(function))
    print("Absoluter Fehler:")
    display(Math(sp.latex(absolute_err,long_frac_ratio=2)))
    print(sp.latex(absolute_err))
    print("Relativer Fehler:")
    display(Math(sp.latex(relative_err,long_frac_ratio=2)))
    print(sp.latex(relative_err))
    return function,absolute_err, relative_err, errProneParamaters

```

Sigma-Abweichungen

```

[7]: def sigma_abweichung(p1, err_p1,p2,err_p2):
    """
    Funktion zum berechnen der Sigma-Abweichugn von zwei Messwerten, oder einem
    ↪ Messwert und einem Literaturwert.
    """
    if err_p1 == 0 and err_p2 == 0:
        raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert
        ↪ fehlerbehaftet sein!")
    else:
        abweichung=Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2+err_p2 ** 2))))

    if abweichung == 0:
        return 0.0, "\\num{0}"

    exp = int(math.floor(math.log10(float(abweichung))))
    if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn
    ↪ 1,2,3 erste signifikante Stelle, dann kommt eine zweite
    ↪ Nachkommastellenposition hinzu
        exp -= 1

```

```

scale = Decimal(10) ** exp
abweichung_round = (abweichung / scale).quantize(Decimal('1'),
↪rounding=ROUND_CEILING) * scale
res = f"\\num{{{abweichung_round}}}"

return float(abweichung_round),res

```

```

[8]: # #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<" + name2
#     elif val1>val2:
#         return name1+">" + name2
#     else:
#         return name1+"=" + name2

#Erstellung von Vergleichstabellen in Latex
def compare_table_array(nam1, nam2, einheiten, val1_array, err1_array,
↪val2_array, err2_array):
    # 1. Tabellenkopf (Hier werden die Strings einmalig eingesetzt)
    output_header = textwrap.dedent(f"""
        \\begin{{table}}[htb]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
        \\toprule
        Messreihe & {nam1}[\\unit{{{einheiten}}}] &
↪{nam2}[\\unit{{{einheiten}}}] & \\text{{abs.Abw.}}[\\unit{{{einheiten}}}] &
↪\\text{{proz.Abw.}}[\\unit{{\\percent}}] & S[\\sigma] \\midrule
    """).strip() # strip macht whitespaces am anfang und ende des strings weg

    table_rows = []

    # 2. Der Body (Schleife läuft über die Werte-Arrays)
    for val1, err1, val2, err2 in zip(val1_array, err1_array, val2_array,
↪err2_array):

        # Deine Berechnungen (bleiben exakt gleich)
        VAL1 = round_to_sigs(val1, err1, r')[2]
        VAL2 = round_to_sigs(val2, err2, r')[2]

        abs_val = np.abs(val1 - val2)
        abs_delta = np.sqrt(err1**2 + err2**2)

        if abs_delta != 0:
            SIGMA = sigma_abweichung(val1, err1, val2, err2)[1]
        else:

```

```

        SIGMA = 0.0

        ABS = round_to_sigs(abs_val, abs_delta, r'')[2]

        rel = abs_val / np.abs(val1) * 100 if val1 != 0 else 0
        rel_delta = rel * np.sqrt((abs_delta / abs_val)**2 + (err1 / val1)**2)
        if abs_val != 0 and val1 != 0 else 0
        REL = round_to_sigs(rel, rel_delta, r'')[2]

        # Eine saubere Zeile nur mit den Werten für LaTeX
        row = f"          & {VAL1} & {VAL2} & {ABS} & {REL} & {SIGMA} \\\\"
        table_rows.append(row)

    body_str = "\\n".join(table_rows)

    # 3. Tabellenfuß
    output_footer = textwrap.dedent(f"""
        \\bottomrule
        \\end{{tabularx}}
        \\label{{table:Vergleich_{nam1}}}
        \\end{{table}}
    """).strip()

    # Alles zusammensetzen
    final_output = f"{output_header}\\n{body_str}\\n{output_footer}"

    print(final_output)

```

```

[ ]: # #Erstellung von Vergleichstabellen in Latex
# def compare_table_withliterature(nam1,nam2,einheiten,val1,err1,val2):
#     VAL1=round_to_sigs(val1,err1,r'')[2]
#     abs=np.abs(val1-val2)
#     abs_delta=np.sqrt(err1**2)
#     if abs_delta!=0:
#         SIGMA=sigma_abweichung(val1,err1,val2,0)[1]
#     else:
#         SIGMA=0.0
#     ABS=round_to_sigs(abs,abs_delta,r'')[2]
#     rel=abs/np.abs(val2)*100
#     rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else
    0
#     REL=round_to_sigs(rel,rel_delta,r'')[2]
#     output = textwrap.dedent(f"""
#         \\begin{{table}}[htb]
#         \\centering
#         \\caption{{}}
#         \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}

```



```

#           \\toprule
#
#           □
#   ↪ {nam1}[\\unit{{{einheiten}}}]&{nam2}[\\unit{{{einheiten}}}]&\\text{{abs.Abw.
#   ↪ }}[\\unit{{{einheiten}}}]&\\text{{proz.Abw.
#   ↪ }}[\\unit{{\\percent}}]&S[\\sigma]\\\\\\\\\\midrule
#           {VAL1}&\\num{{{val2}}}&{ABS}&{REL}&{SIGMA}\\\\\\\\
#           \\bottomrule
#           \\end{{tabularx}}
#           \\label{{table:Vergleich_{nam1}}}
#           \\end{{table}}
#   """)
#   print(output)

# #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
import textwrap
def compare_table(nam1,nam2,einheiten,val1,err1,val2,err2):
    VAL1=round_to_sigs(val1,err1,r'[2]
    if err2!=0:
        VAL2=round_to_sigs(val2,err2,r'[2]
    else:
        VAL2=val2
    abs=np.abs(val1-val2)
    abs_delta=np.sqrt(err1**2+err2**2)
    if abs_delta!=0:
        SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
    else:
        SIGMA=0.0
    ABS=round_to_sigs(abs,abs_delta,r'[2]
    rel=abs/np.abs(val1)*100
    rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else 0
    REL=round_to_sigs(rel,rel_delta,r'[2]
    output = textwrap.dedent(f"""
        \\begin{{table}}[htb]
        \\centering
        \\caption{{}}
        \\begin{{tabularx}}{{\\textwidth}}{{ZZZx}}

```


Absoluter Fehler:

$$\sqrt{C^2 \Delta_R^2 + \Delta_C^2 R^2}$$

$$\sqrt{C^2 \Delta_R^2 + \Delta_C^2 R^2}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_R^2}{R^2} + \frac{\Delta_C^2}{C^2}}$$

$$\sqrt{\frac{\Delta_R^2}{R^2} + \frac{\Delta_C^2}{C^2}}$$

```
[11]: (C*R,
      sqrt(C**2*Delta_R**2 + Delta_C**2*R**2),
      sqrt(Delta_R**2/R**2 + Delta_C**2/C**2),
      [(R, Delta_R), (C, Delta_C)])
```

```
[12]: gff("T_12/ln(2)", "T_12")
```

Funktion:

$$\frac{T_{12}}{\log(2)}$$

$$\frac{T_{12}}{\log(2)}$$

Absoluter Fehler:

$$\frac{\sqrt{\Delta_{T_{12}}^2}}{\log(2)}$$

$$\frac{\sqrt{\Delta_{T_{12}}^2}}{\log(2)}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_{T_{12}}^2}{T_{12}^2}}$$

$$\sqrt{\frac{\Delta_{T_{12}}^2}{T_{12}^2}}$$

```
[12]: (T_12/log(2),
      sqrt(Delta_T_12**2)/log(2),
      sqrt(Delta_T_12**2/T_12**2),
      [(T_12, Delta_T_12)])
```

II Differnetation einer Rechteckspannung

Vorversuch Rauschspektrum

```
[13]: def load_and_save_clean_csv(file_path, skiprowsnumber):
      # 1. Den vollen Pfad zur Originaldatei erstellen
      original_path = Messwerte_path / file_path

      # 2. Die kaputte Datei einlesen
```

```

with open(original_path, "r", encoding="utf-8") as f:
    text = f.read()

    # 3. Den Text bereinigen (Tausender-Kommas entfernen)
    # macht groÙteil richtig: ^ Sucht nur am anfang einer Zeile; (\d+) sucht
↪ erste Zahl 124 bei 124,111.000;
    # , sucht Komma direkt hinter Zahlenblock; (\d+) sucht nächsten Zahlenblock
↪ 111; \. maskierter Punkt (. wäre jegliches Zeichen)
    # r"\1\2. entfernt das Komma zwischen den Gruppen
    sauberer_text = re.sub(r"^(\d+),(\d+)\.", r"\1\2.", text, flags=re.
↪ MULTILINE)
    # fixt kleine ausnahmen, die keine Nachkommastellen haben, weil exakt .0:
↪ 15,625 wurde nicht erkannt und blieb 15,625.
    # ,(\d{3}) erkennt genau die hundertergruppe, die immer 3 Ziffern hat
    sauberer_text = re.sub(r"^(\d+),(\d{3})", r"\1\2,", sauberer_text,
↪ flags=re.MULTILINE)
    # jetzt blieben noch 1(,)111,111.111 Zahlen, also Megahertz freq, deren
↪ erstes Komma wurde in Iteration 1 schon entfernt. Jetzt fehlt also noch das
↪ zweite.
    sauberer_text = re.sub(r"^(\d+),(\d{3})", r"\1\2", sauberer_text, flags=re.
↪ MULTILINE)

    # 4. Den neuen Dateipfad im SELBEN Ordner generieren
    # original_path.parent ist der Ordner (Messwerte_path)
    # original_path.name ist z.B. 'deine_datei.csv'
    neu_datei_path = original_path.parent / f"bereinigt_{original_path.name}" #
↪ fixt den bug für zahlen >10^6, bei denen ., von einer vorherigen sub
↪ geschrieben wurde

    # 5. Den bereinigten Text in die neue Datei schreiben
    with open(neu_datei_path, "w", encoding="utf-8") as f:
        f.write(sauberer_text)

    print(f"Saubere Datei wurde erstellt unter: {neu_datei_path}")

    # 6. Die Daten nun ganz normal aus der neu erstellten Datei laden
    data = np.loadtxt(neu_datei_path, delimiter=",", skiprows=skiprowsnumber,
↪ usecols=(0, 1), unpack=True)

    return data

# --- Anwendung ---
# frequenz, amplitude = load_and_save_clean_csv('deine_datei.csv')

```

```
[14]: plt.close('all')
plt.figure(figsize=(figsize_cm(10.8,6)))
plt.title(r'Rauschspektrum für $f=100$ Hz bis 2 MHz')
plt.xlabel(r'$f$ [Hz]')
plt.ylabel(r'Pegel [dBm]')

f_5,P_5 = load_and_save_clean_csv('5kohm_fspectrum.csv',32)
f_30,P_30 = load_and_save_clean_csv('30kohm_fspectrum.csv',32)

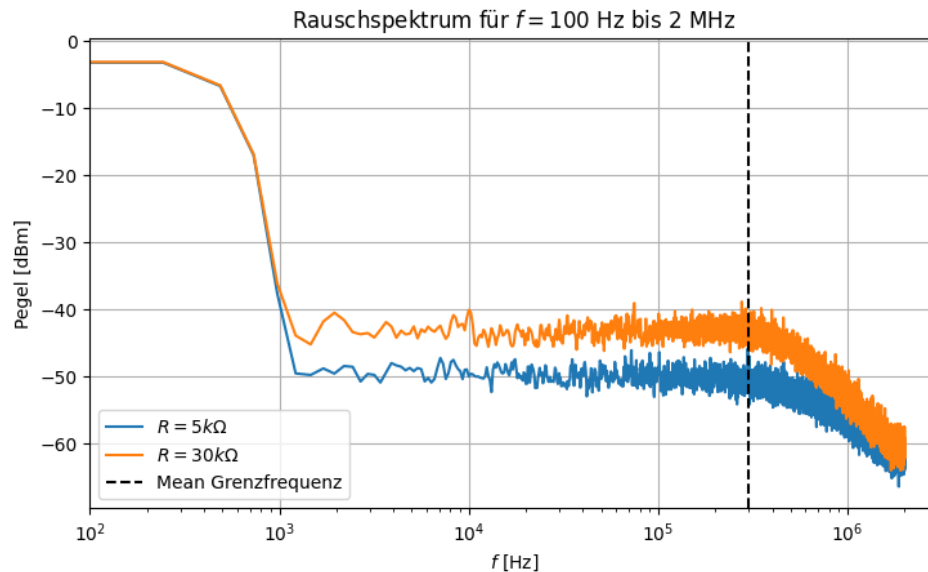
plt.plot(f_5,P_5,label=rf'$R=5k\Omega$')
plt.plot(f_30,P_30,label=rf'$R=30k\Omega$')
plt.axvline(f_5[1230],linestyle='--',color='black',label='Mean Grenzfrequenz')
diff=np.mean(P_5[:1230]-P_30[:1230])
print(diff)
print(f_5[1230])
plt.xlim(100,3e6)
plt.xscale('log')
plt.legend()
plt.grid()
# plt.tight_layout()
plt.savefig(Ausgabe_path/"Rauschpegel_5_30.
    png",format="png",bbox_inches='tight')
plt.show()
```

Saubere Datei wurde erstellt unter: c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Messwerte\243\bereinigt_5kohm_fspectrum.csv

Saubere Datei wurde erstellt unter: c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Messwerte\243\bereinigt_30kohm_fspectrum.csv

-7.0924460479928895

300292.96875



I Frequenzgang

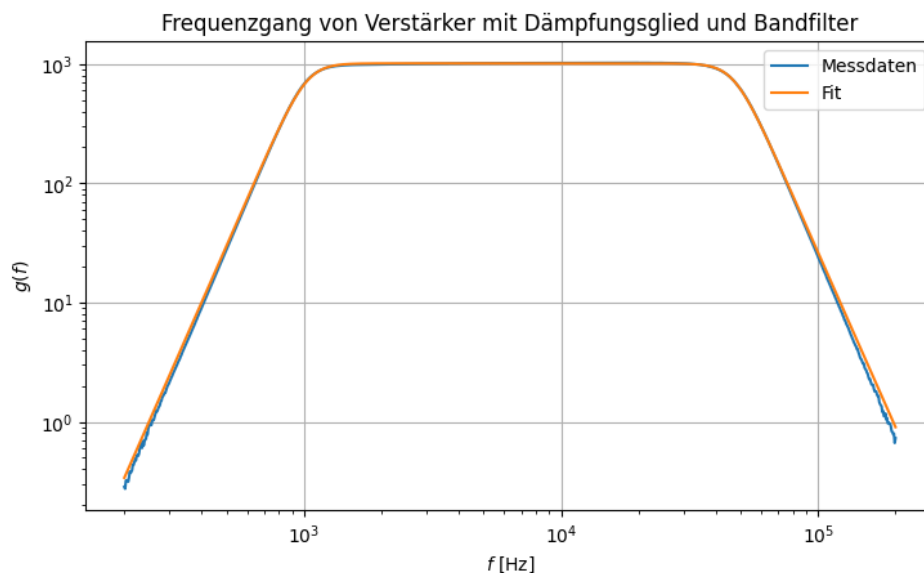
```
[ ]: plt.close('all')
plt.figure(figsize=(figsize_cm(10.8,6)))
plt.title(r'Frequenzgang von Verstärker mit Dämpfungsglied und Bandfilter')
plt.xlabel(r'$f$ [Hz]')
plt.ylabel(r'$g(f)$')

f,P = load_and_save_clean_csv('frequenzgang.csv',17)
D=1e-3
g=P/D
plt.loglog(f,g,label=rf'Messdaten')
# plt.xlim(100,3e6)
# plt.xscale('log')
# plt.yscale('log')
def fit_func(f,V,W1,W2,n1,n2):
    return V/(np.sqrt(1+1/(f/W1)**(2*n1))*np.sqrt(1+(f/W2)**(2*n2)))
p0 = [1000 , 1000 , 50000 , 5 , 5]
popt, pcov = curve_fit(fit_func, f, g ,p0)
plt.loglog(f, fit_func(f, *popt), label='Fit')
for i in range(len(p0)):
    wert,fehler,latex,_=round_to_sigs(popt[i],np.sqrt(pcov[i][i]),einheiten='')
    print(latex)
    print(fehler/wert*100)
plt.legend()
plt.grid()
# plt.tight_layout()
```

```
plt.savefig(Ausgabe_path/"Frequenzgang_Fit.
↳png",format="png",bbox_inches='tight')
plt.show()
```

Saubere Datei wurde erstellt unter: c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Messwerte\243\bereinigt_frequenzgang.csv

```
\num{1010.1(0.4)}
0.039600039600039605
\num{1024.2(0.8)}
0.07810974419058778
\num{47190(40)}
0.0847637211273575
\num{4.896(0.019)}
0.3880718954248366
\num{4.861(0.019)}
0.39086607693890146
```



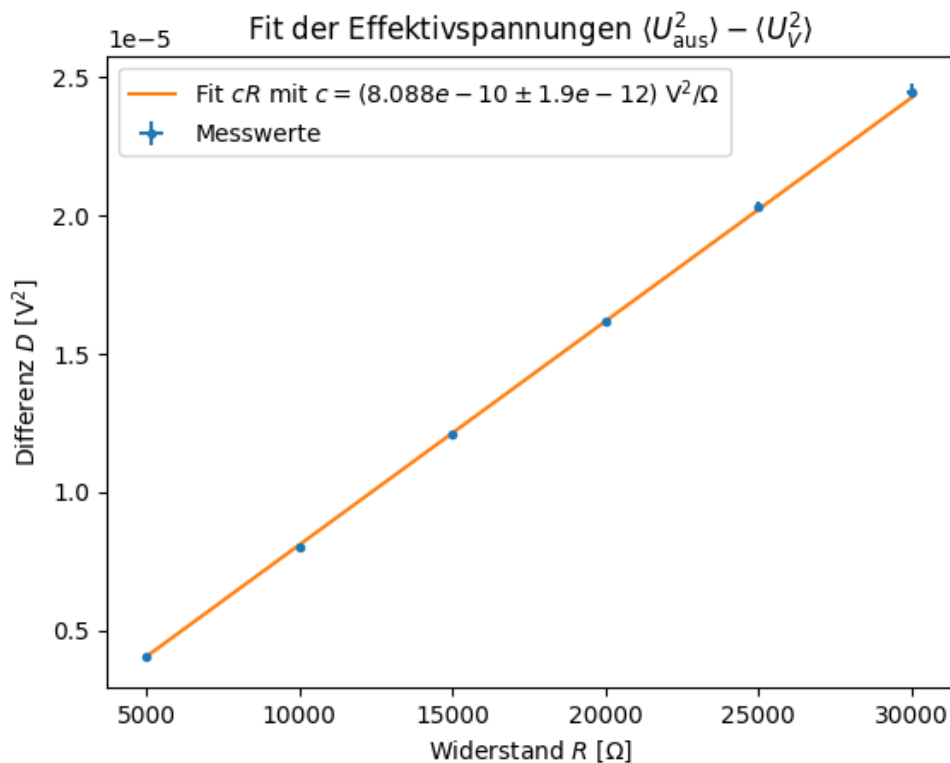
```
[ ]: def fit_func_square(f,V,W1,W2,n1,n2):
    return fit_func(f,V,W1,W2,n1,n2)**2
B=integrate.quad(fit_func_square, f[0], f[-1], args=tuple(popt))
B,Delta_B,_,latexB=round_to_sigs(B[0],B[0]*0.02,r'\hertz')
print(latexB)
# print(f'Das Integral beträgt: {round_to_sigs(B,B*0.02,r'\hertz')[2]}')
```

\qty{47900000000(1000000000)}{\hertz}

```
[65]: R=np.array([5e3, 10e3 , 15e3 , 20e3 , 25e3 , 30e3]) #Eingabe der Widerstände
Delta_R=R*0.005
U_aus=np.array([2.4436,3.1588,3.7474,4.2562,4.7182,5.1399])*1e-3
Delta_U_aus=np.array([0.0098,0.0132,0.0143,0.0158,0.0184,0.027])*1e-3
U_V=1.3958*1e-3
Delta_U_V=0.0116*1e-3
D=U_aus**2-U_V**2 #Berechnung Differenz
Delta_D=np.sqrt((2*Delta_U_aus*U_aus)**2 + (2*Delta_U_V*U_V)**2)

plt.close('all')
plt.errorbar(R, D, yerr=Delta_D, xerr= Delta_R, fmt='.',label='Messwerte')
# plt.axis([0,3.5e4,0,2.5e-5])
def linear(x,c): #Definition einer linearen Funktion als Fit
    return c*x
popt1, pcov1 = curve_fit(linear, R, D , sigma=Delta_D) #Durchführung des Fits
c,Delta_c,_,latexc=round_to_sigs(popt1[0],np.
    ↪sqrt(pcov1[0][0]),r'\Volt\squared\per\Omega')
plt.plot(R,linear(R,*popt1),label = rf'Fit $cR$ mit $c=({c})\pm{Delta_c})\backslash$;
    ↪\mathrm{{V}}^2/\Omega$')
print(latexc)
plt.title(r'Fit der Effektivspannungen $\langle U_{\text{aus}}^2 \rangle - \langle U_V^2 \rangle$')
plt.xlabel(r'Widerstand $R$ $[\Omega]$')
plt.ylabel(r'Differenz $D$ $[\text{V}^2]$')
plt.legend()
plt.savefig(Ausgabe_path/"Differenz_D.png",format="png",bbox_inches='tight')
plt.show()
```

$\sqrt{8.088\text{E-}10(1.9\text{E-}12)} \backslash \text{Volt} \backslash \text{squared} \backslash \text{per} \backslash \Omega$



```
[ ]: chisquare=np.sum(((linear(R,*popt1)-D)**2/Delta_D**2))
dof=5 #degrees of freedom
chisquare_red=chisquare/dof
print(chisquare)
print(chisquare_red)

from scipy.stats import chi2
prob=round(1-chi2.cdf(chisquare,dof),4)*100
print("Wahrscheinlichkeit="+str(prob)+"%")
```

```
1.4947555699945816
0.29895111399891633
Wahrscheinlichkeit=91.36999999999999%
```

```
[ ]: T = 25.1 + 273.15
Delta_T = 0.5
k = c/(4*T*B)
Delta_k_syst = k * Delta_B/B
Delta_k_stat = k*np.sqrt(Delta_T**2/T**2 + Delta_c**2/c**2)
_,Delta_k_syst,_,_,latexk=round_to_sigs(k,Delta_k_syst,'')
k,Delta_k_stat,_,_=round_to_sigs(k,Delta_k_stat,r'\joule\per\kelvin')
```

```
print('k=',k, Delta_k_stat, Delta_k_syst)
```

```
k= 1.415e-23 5e-26 3e-25
0.021201413427561835
```

```
[64]: compare_table(r'k_{\text{exp}}',r'k_{\text{lit}}',r'\joule\per\kelvin',1.415,0.03,1.
↪380649,0)
```

```
\begin{table}[htb]
\centering
\caption{}
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
k_{\text{exp}}[\unit{\joule\per\kelvin}]&k_{\text{lit}}[\unit{\joule\per\kel
vin}]&\text{abs.Abw.}[\unit{\joule\per\kelvin}]&\text{proz.Abw.}[\unit{\percent}]
&S[\sigma]\\\midrule
\num{1.42(0.03)}&1.380649&\num{0.03(0.03)}&\num{2.4(2.2)}&\num{1.2}\\\
\bottomrule
\end{tabularx}
\label{table:Vergleich_k_{\text{exp}}}
\end{table}
```

```
[51]: gff("c/(4*T*B)", "c,T")
```

Funktion:

$$\frac{c}{4BT}$$

$\frac{c}{4BT}$

Absoluter Fehler:

$$\frac{1}{4} \sqrt{\frac{1}{B^2 T^4} (\Delta_T^2 c^2 + \Delta_c^2 T^2)}$$

$$\frac{\sqrt{\frac{\Delta_T^2}{B^2 T^4} c^2 + \Delta_c^2 T^2}}{4}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_T^2}{T^2} + \frac{\Delta_c^2}{c^2}}$$

$$\sqrt{\frac{\Delta_T^2}{T^2} + \frac{\Delta_c^2}{c^2}}$$

```
[51]: (c/(4*B*T),
sqrt((Delta_T**2*c**2 + Delta_c**2*T**2)/(B**2*T**4))/4,
sqrt(Delta_T**2/T**2 + Delta_c**2/c**2),
[(c, Delta_c), (T, Delta_T)])
```